

AI 에이전트

전문가 양성 과정

OOP



Contents

1. OOP

- 객체
- OOP

2. 파일 입출력

- 파일 읽기
- 파일 경로
- 파일 쓰기
- CSV
- JSON

3. 예외 처리

제 1장.

OOP

1-1 객체

class

붕어빵을 찍어내는 틀
(객체를 만들기 위한 설계도)

객체

그 틀로 만들어진 붕어빵
(class로 만든 실체)

객체

데이터(속성/Attribute)와 기능(함수/Method)를 함께 묶어놓은 것

👉 관련 있는 데이터와 기능을 하나로 묶어 관리하는 방법

객체

- langChain RecursiveCharacterTextSplitter ([링크](#))
 - RecursiveCharacterTextSplitter 라는 class 가져오기
 - 속성 설정
 - 함수 실행

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text = "LangChain은 LLM 애플리케이션을 만들기 위한 프레임워크입니다."

# class를 사용해서 객체 생성
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=50
)

# 객체의 메서드 사용
chunks = text_splitter.split_text(text)

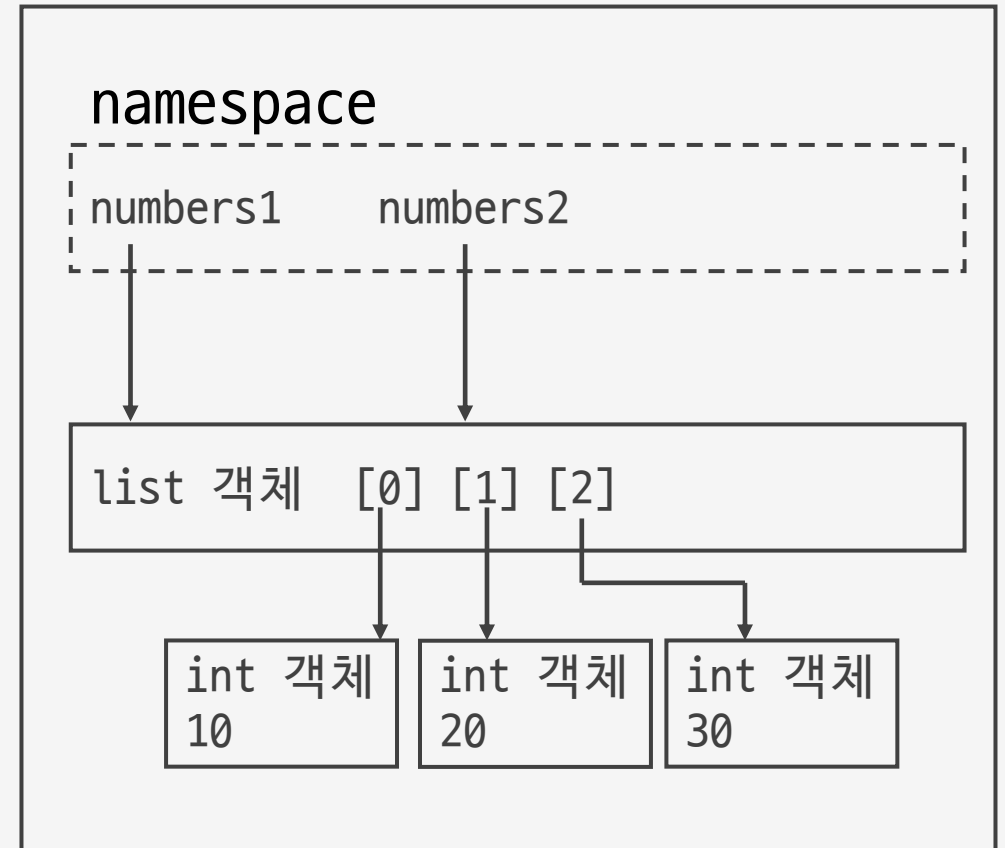
print(chunks)
```

객체

- 객체 (Object) : 데이터(속성)와 기능(함수)을 함께 묶어놓은 것
- Python에서는 숫자, 문자열, 리스트, 딕셔너리 모두 객체

```
numbers1 = [10, 20, 30]
numbers2 = numbers1

numbers2.append(40)
```



객체

- 객체 (Object) : 데이터(속성)와 기능(함수)을 함께 묶어놓은 것
- Python에서는 숫자, 문자열, 리스트, 딕셔너리 모두 객체

```
message = 'hello world'

print(message.upper()) # HELLO WORLD
```

- 👉 "hello world"는 str 클래스에서 만들어진 문자열 객체이고, message는 그 문자열 객체를 가리키는 변수.
.upper()는 문자열 객체가 사용할 수 있는 기능, 즉 메서드이다.

객체

- class / 객체 생성 실습 (1/8)
 - class : 클래스를 만들 때 사용하는 키워드
 - User : 클래스 이름 (보통 대문자로 시작)

```
class User:  
    pass
```

객체

- class / 객체 생성 실습 (2/8)
 - class 속성 정의
 - `__init__` : 객체가 만들어질 때 자동으로 실행되는 초기 설정 함수 (생성자 함수, 생성자 메서드)
 - `self` : 객체 자기 자신을 가리키는 이름, 파이썬에서 관례적으로 사용하는 약속

```
class User:
    def __init__(self, name, age, job):
        self.name = name
        self.age = age
        self.job = job
```

객체

- class / 객체 생성 실습 (3/8)
 - 객체 생성 시 속성으로 자동 저장

```
user_1 = User("Jay", 30, "Student")

print(f"이름: {user_1.name}") # 이름: Jay
print(f"나이: {user_1.age}") # 나이: 30
print(f"직업: {user_1.job}") # 직업: Student
```

```
user_2 = User("Alice", 28, "AI Engineer")

print(f"이름: {user_2.name}") # 이름: Alice
print(f"나이: {user_2.age}") # 나이: 28
print(f"직업: {user_2.job}") # 직업: AI Engineer
```

객체

- class / 객체 생성 실습 (4/8)
 - 함수 정의
 - self : 객체 자기 자신을 가리키는 이름, 파이썬에서 관례적으로 사용하는 약속

```
class User:
    def __init__(self, name, age, job):
        self.name = name
        self.age = age
        self.job = job

    def introduce(self):
        print(f"안녕하세요! 제 이름은 {self.name}입니다.")
```

객체

- class / 객체 생성 실습 (5/8)
 - 객체 생성 및 함수 실행

```
user_1 = User("Jay", 30, "Student")
```

```
user_1.introduce()
```

```
# 안녕하세요! 제 이름은 Jay입니다.
```

객체

- class / 객체 생성 실습 (6/8)
 - 객체 생성 및 함수 실행

```
user_2 = User("Alice", 28, "AI Engineer")  
  
user_2.introduce()           # 안녕하세요! 제 이름은 Alice입니다.
```

객체

- class / 객체 생성 실습 (7/8)
 - 속성 수정 함수 정의

```
class User:
    ...
    def introduce(self):
        print(f"안녕하세요! 제 이름은 {self.name}입니다.")

    def have_birthday(self):
        self.age += 1
```

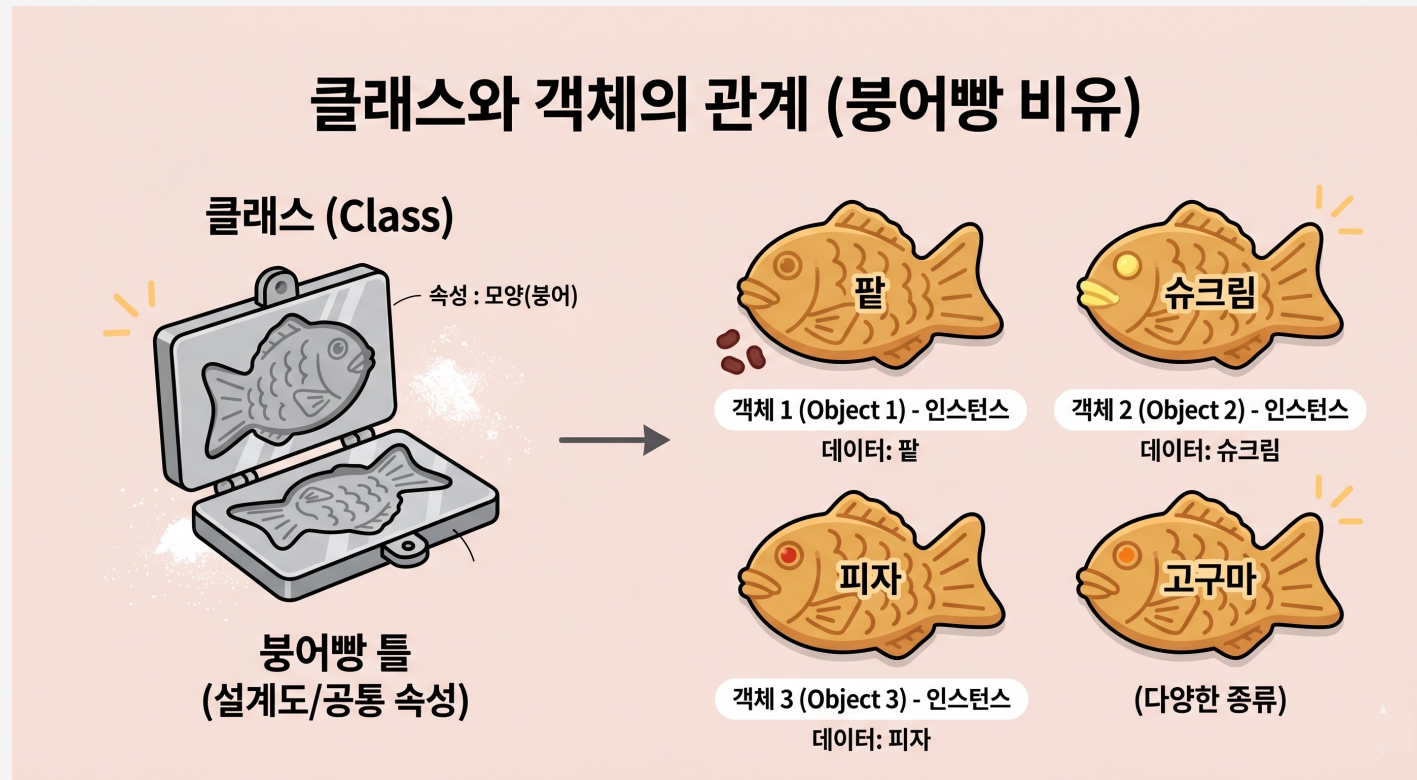

객체

- class / 객체 생성 실습 (8/8)
 - 속성 수정 함수 정의

```
user_1 = User("Jay", 30, "Student")
print(f"나이: {user_1.age}") # 나이: 30
user_1.have_birthday()
print(f"나이: {user_1.age}") # 나이: 31
```

객체

- class : 객체를 만들기 위한 설계도
- 객체 : class로 만든 데이터



객체

- 딕셔너리 vs 객체
 - 딕셔너리 : 데이터를 담는 컨테이너
 - 객체 : 데이터와 기능을 함께 가진 실체

```
user = {  
    "name": "Jay",  
    "age": 30  
}  
  
print(user["name"])
```

```
class User:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
user = User("Jay", 30)  
  
print(user.name)
```

객체

- 딕셔너리 vs 객체
 - 객체는 단순히 데이터를 한 번 담고 끝나는 게 아니라, 자기 안에 현재 값을 저장하고 있다가 메서드를 통해 그 값을 바꿀 수 있다.

```
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def birthday(self):
        self.age += 1

user = User("Jay", 30)

print(user.age) # 30

user.birthday()

print(user.age) # 31
```

객체

- 객체 (Object) : 데이터(속성)와 기능(함수)을 함께 묶어놓은 것
- Python에서는 숫자, 문자열, 리스트, 딕셔너리 모두 객체

```
message = 'hello world'

print(message.upper()) # HELLO WORLD
```

- 👉 message는 문자열 객체이고, .upper()는 그 객체가 가진 기능
- 객체는 단순히 값만 담는 것이 아니라, 그 값과 관련된 행동도 함께 가질 수 있음

'hello world' .upper()

문자열.대문자 변환()

객체.기능()

👉 'hello world' 는 str이라는 클래스의 객체

[1, 2, 3, 4, 5].sort()

리스트.정렬()

객체.기능()

👉 [1,2,3,4,5]는 list라는 클래스의 객체

(복습) 내장 함수

- `map()`
 - 여러 데이터에 같은 함수를 한 번씩 적용할 때 사용
 - `map(함수, 반복 가능한 객체)`
 - `map()`의 결과는 바로 리스트 x, 필요하면 `list()`로 감싸서 확인

```
numbers = [1, 2, 3, 4, 5]
result = []
for number in numbers:
    result.append(number ** 2)
print(result)
```



```
def square(number):
    return number ** 2

numbers = [1, 2, 3, 4, 5]
result = list(map(square, numbers))
print(result)
```


(복습) 내장 함수

- `map()` / `filter()`
 - `map` 객체, `filter` 객체를 `list` 객체로 바꾸기 위함

```
def square(number):  
    return number ** 2  
  
numbers = [1, 2, 3, 4, 5]  
  
result = map(square, numbers)  
print(type(result)) # <class 'map'>  
  
result = filter(square, numbers)  
print(type(result)) # <class 'filter'>
```

(복습) 내장 함수

- `map()` / `filter()`
 - `map`, `filter` 객체를 `list` 객체로 바꾸기 위함
 - `map`, `filter` 객체는 저장된 리스트가 아니라 한 번 지나가면 끝나는 객체

```
buy_trades = filter(lambda trade: trade["type"] == "buy", trades)

buy_stock_names = list(map(lambda trade: trade["name"], buy_trades))
payment_amounts = list(
    map(lambda trade: trade["price"] * trade["quantity"] + fee, buy_trades)
)

print(f"매수 종목: {buy_stock_names}") # ['삼성전자', '네이버', '현대차']
print(f"매수 결제 금액: {payment_amounts}") # []
```

1-2 OOP

OOP

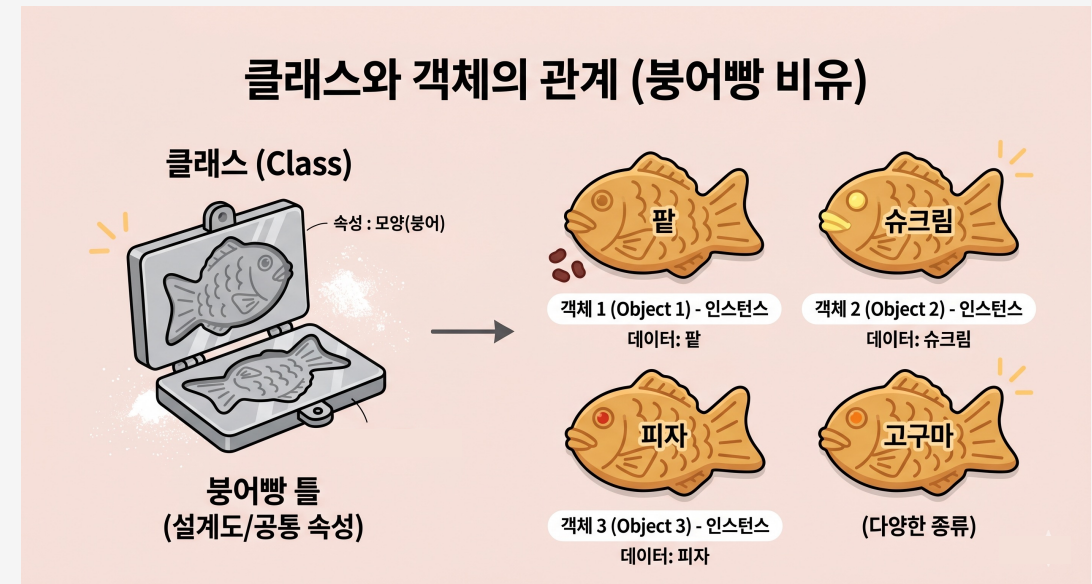
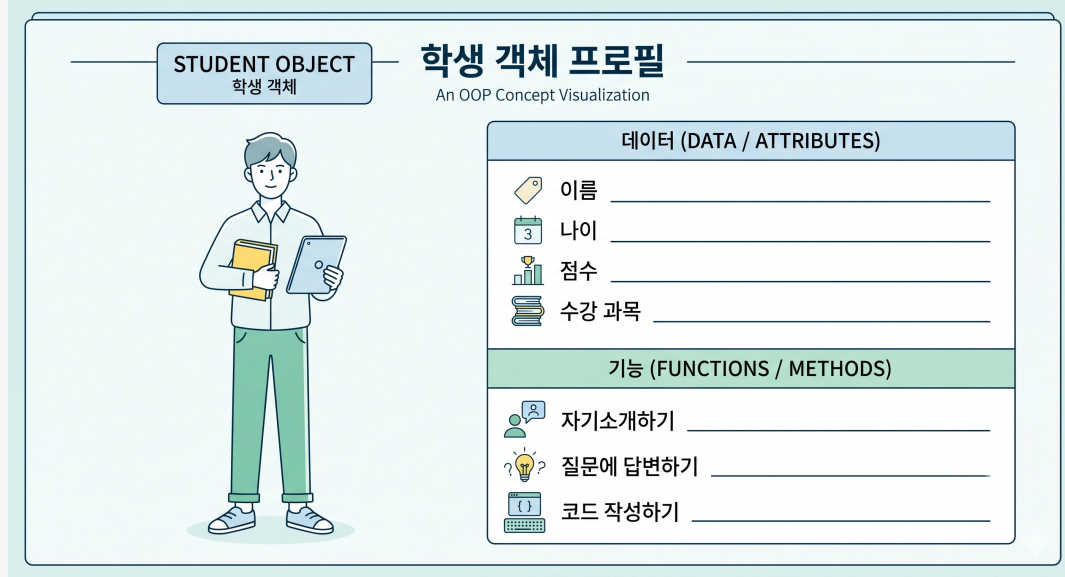
Object-Oriented Programming

객체 지향 프로그래밍

→ 프로그램을 객체들의 조합으로 설계하는 방식

OOP

- Object-Oriented Programming
 - 데이터와 기능을 객체 단위로 묶어서 관리
 - 큰 프로그램을 더 구조적으로 만들 수 있음



OOP

- 쇼핑몰 시스템을 구축한다면???
- User (사용자 객체)
 - 데이터 (Attributes)
 - user_id: 회원 고유 아이디
 - name: 이름
 - email: 이메일 주소
 - shipping_address: 배송지 주소
 - point: 보유 마일리지/포인트
 - 기능 (Methods)
 - login(): 로그인하기
 - update_profile(): 회원 정보(주소 등) 수정하기
 - add_to_cart(product): 원하는 상품을 장바구니에 담기
 - leave_review(product): 구매한 상품에 리뷰 남기기

OOP

- 쇼핑몰 시스템을 구축한다면???
- **Product (상품 객체)**
 - 데이터 (Attributes)
 - product_id: 상품 고유 번호
 - name: 상품명
 - price: 가격
 - stock: 남은 재고 수량
 - category: 상품 카테고리 (의류, 가전 등)
 - 기능 (Methods)
 - get_details(): 상품 상세 정보 보여주기
 - decrease_stock(amount): 상품이 팔렸을 때 재고 감소시키기
 - increase_stock(amount): 환불이나 입고 시 재고 증가시키기
 - apply_discount(percent): 할인율 적용하여 가격 변동하기

OOP

- 쇼핑몰 시스템을 구축한다면???
- Order (주문 객체)
 - 데이터 (Attributes)
 - order_id: 주문 고유 번호
 - buyer: 주문한 사람 (User 객체 연결)
 - items: 주문한 상품 목록 (Product 객체들의 배열/리스트)
 - total_amount: 총 주문 결제 금액
 - order_status: 주문 상태 (결제대기, 상품준비중, 배송중, 배송완료)
 - 기능 (Methods)
 - calculate_total(): 담긴 상품들의 가격을 합산하여 총액 계산하기
 - update_status(new_status): 배송 출발 등 주문 상태 변경하기
 - cancel_order(): 주문 취소하기

OOP

- 쇼핑몰 시스템을 구축한다면???
- **Payment (결제 객체)**
 - 데이터 (Attributes)
 - payment_id: 결제 승인 번호
 - order: 결제할 주문 내역 (Order 객체 연결)
 - payment_method: 결제 수단 (신용카드, 무통장입금, 카카오페이 등)
 - is_successful: 결제 성공 여부 (True/False)
 - paid_date: 결제 완료 일시
 - 기능 (Methods)
 - process_payment(): PG사(결제대행사)와 통신하여 실제 결제 진행하기
 - issue_receipt(): 결제 영수증 발행하기
 - refund_payment(): 주문 취소 시 결제 금액 환불 처리하기

OOP

- 객체 지향 프로그래밍 장점
 - 프로젝트가 커졌을 때 코드를 관리하기 편하게 만들기 위함

```
class Product:
    def __init__(self, product_id, name, price, stock):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock = stock

    def get_details(self):
        print(f"[{self.product_id}] {self.name} | 가격: {self.price:,}원 | 남은 재고: {self.stock}개")

    def decrease_stock(self, amount):
        if self.stock >= amount:
            self.stock -= amount
            print(f"'{self.name}' {amount}개 판매 완료! (현재 재고: {self.stock}개)")
        else:
            print(f"'{self.name}'의 재고가 부족합니다! (현재 재고: {self.stock}개, 요청 수량: {amount}개)")

    def apply_discount(self, percent):
        discount_amount = self.price * (percent / 100)
        self.price = int(self.price - discount_amount)
        print(f"{percent}% 할인 적용! '{self.name}'의 가격이 {self.price:,}원으로 인하되었습니다.")
```

OOP

- OOP 핵심 개념
 - 추상화 : 현실 세계를 프로그램에 반영 → 복잡한 대상을 단순하게 모델링 가능
 - 상속 : 클래스는 클래스 간 상속 가능 → 코드 재사용을 통한 중복 감소
 - 다형성 : 같은 이름의 메서드가 객체에 따라 다르게 동작 → 코드 유연성 증가
 - 캡슐화 : 객체 내부의 데이터와 기능은 외부에서 함부로 건드리지 않음 → 데이터 보호

OOP

추상화

- 현실 세계를 프로그램에 반영 → 복잡한 대상을 단순하게 모델링 가능

STUDENT OBJECT
 학생 객체

학생 객체 프로필
 An OOP Concept Visualization



데이터 (DATA / ATTRIBUTES)	
	이름 _____
	나이 _____
	점수 _____
	수강 과목 _____
기능 (FUNCTIONS / METHODS)	
	자기소개하기 _____
	질문에 답변하기 _____
	코드 작성하기 _____

```

class Product:
    def __init__(self, product_id, name, price, stock):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock = stock

    def get_details(self):
        print(f"[{self.product_id}] {self.name} | 가격: {self.price:,}원 | 남은 재고: {self.stock}개")

    def decrease_stock(self, amount):
        if self.stock >= amount:
            self.stock -= amount
            print(f"'{self.name}' {amount}개 판매 완료! (현재 재고: {self.stock}개)")
        else:
            print(f"'{self.name}'의 재고가 부족합니다! (현재 재고: {self.stock}개, 요청 수량: {amount}개)")

    def apply_discount(self, percent):
        discount_amount = self.price * (percent / 100)
        self.price = int(self.price - discount_amount)
        print(f"{percent}% 할인 적용! '{self.name}'의 가격이 {self.price:,}원으로 인하되었습니다.")
  
```

OOP

- 상속
 - 클래스는 클래스 간 상속 가능
- 코드 재사용을 통한 중복 감소

```
class Animal:
    def speak(self):
        print("소리를 냅니다.")

class Dog(Animal):
    def bark(self):
        print("멍멍!")

class Cat(Animal):
    def meow(self):
        print("야옹!")

dog = Dog()
cat = Cat()

dog.speak()    # 소리를 냅니다.
dog.bark()     # 멍멍!
cat.speak()    # 소리를 냅니다.
cat.meow()     # 야옹!
```

OOP

- 상속
 - 다중 상속도 가능
 - 왼쪽부터 순서가 있고,
메서드 이름이 겹칠 때 먼저 적은
클래스의 메서드가 실행

```
class Flyer:
    def fly(self):
        print("하늘을 납니다.")

class Swimmer:
    def swim(self):
        print("물에서 헤엄칩니다.")

class Duck(Flyer, Swimmer):
    pass

duck = Duck()

duck.fly()
duck.swim()
```

OOP

- 상속
 - 다중 상속도 가능
 - 왼쪽부터 순서가 있고,
메서드 이름이 겹칠 때 먼저 적은
클래스의 메서드가 실행

```
class A:
    def hello(self):
        print("A의 hello")

class B:
    def hello(self):
        print("B의 hello")

class Child(A, B):
    pass

child = Child()
child.hello() # A의 hello
```

OOP

- 다형성
 - 같은 이름의 메서드가 객체에 따라 다르게 동작 → 코드 유연성 증가

```
class Dog:
    def speak(self):
        return "멍멍"

class Cat:
    def speak(self):
        return "야옹"

animals = [Dog(), Cat()]

for animal in animals:
    print(animal.speak())
```


OOP

- 캡슐화
 - 객체 내부의 데이터와 기능은 외부에서 함부로 건드리지 않음 → 데이터 보호
 - 계좌 잔액 `balance`를 외부에서 직접 마음대로 바꾸지 못하게 숨기고, `deposit()` 같은 정해진 메서드를 통해서만 바꾸게 만든 코드
 - `__balance` : 외부에서 직접 수정하려 하면 에러 발생

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount <= 0:
            print("입금액은 0보다 커야 합니다.")
            return
        self.__balance += amount

    def withdraw(self, amount):
        if amount <= 0:
            print("출금액은 0보다 커야 합니다.")
            return

        if amount > self.__balance:
            print("잔액이 부족합니다.")
            return

        self.__balance -= amount

    def get_balance(self):
        return self.__balance

account = BankAccount(10000)
account.deposit(5000)
account.withdraw(3000)
print(account.get_balance())
```

제 2장.

파일 입출력

2-1 파일 읽기

파일 읽기

- 사용자 입출력
 - 터미널 데이터 입력 및 출력
 - 프로그램에서 데이터를 매번 직접 입력하기는 어려움
 - 프로그램이 종료되어도 데이터를 남겨야 하는 상황
 - 데이터를 파일로 남기고(저장하고), 파일을 읽어 입력 받기

```
number = input("숫자를 입력하세요: ")  
  
print(f"입력한 숫자 : {number}")
```

파일 읽기

- 파일 입출력
 - 사용자 입력값을 저장할 때
 - 프로그램 실행 결과를 기록할 때
 - 설정값을 파일에서 불러올 때
 - 로그를 남길 때
 - 데이터 분석용 파일을 읽을 때

```
user_input.txt  
result.pdf  
config.json  
log.txt  
data.csv
```

구분	의미	예시
파일 읽기	파일에 저장된 내용을 가져오기	read()
파일 쓰기	파일에 내용을 저장하기	write()

파일 읽기

open(파일 경로, 모드)

Python에서 파일을 다룰 때는 open() 함수를 사용

파일 읽기

- 파일 입력
 - "memo.txt": 열고 싶은 파일 경로
 - "r": 읽기 모드
 - encoding="utf-8": 문자 인코딩 방식
 - read(): 파일 내용 읽기
 - close(): 파일 닫기

```
file = open("memo.txt", "r", encoding="utf-8")  
  
content = file.read()  
file.close()  
  
print(content)
```

파일 읽기

- 파일 모드

모드	의미	설명
"r"	읽기	파일 내용을 읽기
"w"	쓰기	새로 파일 작성 (기존 내용은 사라짐)
"a"	추가	기존 내용 뒤에 이어서 작성
"x"	생성	새 파일 생성. 이미 있으면 오류

파일 읽기

- 파일 입력
 - "memo.txt": 생성할 파일 경로
 - "w": 쓰기 모드
 - encoding="utf-8": 문자 인코딩 방식
 - write(): 파일 내용 쓰기
 - close(): 파일 닫기

```
file = open("memo.txt", "w", encoding="utf-8")  
  
file.write("안녕하세요.\n")  
file.write("Python 파일 입출력을 배우고 있습니다.\n")  
  
file.close()
```

파일 읽기

- `encoding="utf-8"`
 - 유니코드 문자를 바이트로 저장하는 대표적인 인코딩 방식
 - 전 세계 문자, 한글, 이모지 등을 표현할 수 있음
 - 현재 웹, API, 파일 저장에서 가장 널리 사용됨

문자 종류	UTF-8 저장 크기
영어, 숫자	보통 1바이트
한글	보통 3바이트
이모지	보통 4바이트

```
print("A".encode("utf-8")) # b'A'  
print("가".encode("utf-8")) # b'\xea\xb0\x80'  
print("😊".encode("utf-8")) # b'\xf0\x9f\x99\x82'
```

(복습) 유니코드

- 인코딩
 - Unicode는 문자에 번호를 붙인 표준 (16진수)
 - 하지만 컴퓨터는 데이터를 바이트로 저장
 - 인코딩 Encoding : 문자 번호를 바이트로 바꾸는 방식
 - 문자 → 사람 눈에 보이는 글자 (가)
 - 유니코드 → 문자에 붙은 주민등록번호 (U+AC00)
 - 인코딩 → 그 번호를 컴퓨터 파일에 저장하는 포장 방식 (ea b0 80)
- UTF-8 : 대표적인 인코딩 방식

(복습) 유니코드

- 인코딩
 - 문자 : ㄱ
 - 유니코드 : U+3131 (16진수)
 - UTF-8 : E3 84 B1 (3바이트)
 - 2진수 :
 - E3 → 11100011 (E : 1110 , 3 : 0011)
 - 84 → 10000100
 - B1 → 10110001
 - 총 크기:
 - 3바이트 = 24비트

파일 읽기

- `close()`
 - 파일을 열면 사용이 끝난 뒤 닫아야 함
 - 파일을 닫지 않으면...
 - 파일이 계속 사용 중인 상태로 남을 수 있음
 - 저장이 제대로 끝나지 않을 수 있음
 - 여러 파일을 다룰 때 자원 낭비가 생길 수 있음
- 👉 Python에서는 보통 `with`를 사용

파일 읽기

with

사용이 끝난 자원을 자동으로 정리해주는 문법

파일 읽기

with open(파일 경로, 모드)

파일을 자동으로 닫아주는 문법

실무에서 더 자주 사용하는 방식

파일 읽기

- with open() 사용 파일 읽기
 - close() 없이 안전하게 파일 읽기 가능

```
with open("memo.txt", "r", encoding="utf-8") as file:  
    content = file.read()  
  
print(content)
```


파일 읽기

- with open() 사용 파일 쓰기
 - close() 없이 안전하게 파일 쓰기 가능

```
with open( " new_memo.txt", "w", encoding="utf-8") as file:  
    file.write("첫 번째 줄입니다.\n")  
    file.write("두 번째 줄입니다.\n")
```

파일 읽기

- with open() + a 모드
 - close() 없이 안전하게 파일 쓰기 가능
 - a 모드로 기존 파일에 내용 추가

```
with open("new_memo.txt", "a", encoding="utf-8") as file:  
    file.write("세 번째 줄입니다.\n")
```

파일 읽기

- for 반복문으로 파일 한 줄씩 읽기
 - 파일을 줄 단위로 처리할 때 for문을 많이 사용
 - 파일 객체는 for 반복문과 함께 사용 시 자동으로 줄바꿈을 인식해 한 줄씩 꺼냄

```
with open("new_memo.txt", "r", encoding="utf-8") as file:  
    for line in file:  
        print(line)
```

TODO List

파이썬 공부

API 예습

파일 읽기

- 줄바꿈 제거
 - repr() : 문자열 안에 숨어 있는 문자들을 출력하는 내장 함수
 - <https://docs.python.org/ko/3/library/functions.html>

```
with open("data/memo.txt", "r", encoding="utf-8") as file:  
    for line in file:  
        print(repr(line))
```

```
# '안녕하세요.\n'  
# 'Python 파일 입출력을 배우고 있습니다.\n'
```

파일 읽기

- 줄바꿈 제거
 - `strip()` : 문자열 양쪽의 공백과 줄바꿈을 제거

```
with open("new_memo.txt", "r", encoding="utf-8") as file:  
    for line in file:  
        line = line.strip()  
        print(line)
```

파일 읽기

- read() / readline()
 - read() : 파일 전체를 한 번에 읽기 (파일 크기가 작을 때 사용하기 좋음)
 - readline() : 한 줄씩 읽기 (호출할 때마다 다음 줄을 읽음)

```
with open("memo.txt", "r", encoding="utf-8") as file:  
    content = file.read()  
  
print(content)
```

```
with open("data/memo.txt", "r", encoding="utf-8") as file:  
    line1 = file.readline()  
    line2 = file.readline()  
  
print(line1) # 안녕하세요.  
print(line2) # Python 파일 입출력을 배우고 있습니다.
```

파일 읽기

- `read()` / `readline()`
- `readlines()` : 각 줄이 리스트의 요소로 병합

```
with open("new_memo.txt", "r", encoding="utf-8") as file:
    lines = file.readlines()

print(lines)
# ['첫 번째 줄입니다.\n', '두 번째 줄입니다.\n', '세 번째 줄입니다.\n']
```

2-2 파일 경로

파일 경로

- 파일을 열 때는 파일 경로를 지정해야 함
- 경로의 종류
 - 상대 경로 : 현재 실행 위치를 기준으로 한 경로 (memo.txt , data/memo.txt)
 - 절대 경로 : 컴퓨터 전체 위치를 기준으로 한 경로 (C:/Users/Gangnam/Desktop/memo.txt)
- Windows 경로 작성 시 주의 사항
 - Windows 경로에는 역슬래시 \가 들어감 (C:\Users\Jay\Desktop\memo.txt)
 - \n, \t 등이 escape sequence로 해석될 수 있어서 문제가 생길 수 있음
 - C:\Users\nomad\Desktop\test.txt

(복습) 기초 문법

- 자료형 (Data Type)
 - 문자열 (str) : 영어, 한글과 같은 모든 문자
 - Escape Sequence : 문자열 안에서 역슬래시 (\) 사용해 특수한 의미를 가지는 문자 표현

표현	의미	예시
\n	줄바꿈	"Hello\nPython"
\t	탭 간격	"name\tage"
\"	큰따옴표 출력	"\"Hello\""
\'	작은따옴표 출력	'It\'s'
\\	역슬래시 출력	"C:\\Users"

(복습) 모듈

- Python 기본 모듈
 - <https://docs.python.org/3/py-modindex.html>

<code>__future__</code>	Future statement definitions
<code>__main__</code>	The environment where top-level code is run. Covers command-line interfaces, import-time behavior, and <code>__name__ == '__main__'`.</code>
<code>_thread</code>	Low-level threading API.
<code>_tkinter</code>	A binary module that contains the low-level interface to Tcl/Tk.
a	
<code>abc</code>	Abstract base classes according to :pep: `3119`.
<code>aifc</code>	Deprecated: Removed in 3.13.
<code>annotationlib</code>	Functionality for introspecting annotations
<code>argparse</code>	Command-line option and argument parsing library.
<code>array</code>	Space efficient arrays of uniformly typed numeric values.
<code>ast</code>	Abstract Syntax Tree classes and manipulation.
<code>asyncio</code>	Deprecated: Removed in 3.12.
<code>asynchat</code>	Asynchronous I/O.
<code>asyncore</code>	Deprecated: Removed in 3.12.
<code>atexit</code>	Register and execute cleanup functions.
<code>audioop</code>	Deprecated: Removed in 3.13.

모듈	용도
math	수학 계산
random	랜덤 값 생성
datetime	날짜와 시간 처리
os	운영체제 기능 사용
json	JSON 데이터 처리
pathlib	파일, 폴더의 경로를 객체로 처리

파일 경로

- pathlib 라이브러리
 - Path는 파일 경로를 문자열이 아니라 경로 객체로 다루게 해주는 도구
 - Path() : 터미널에서 실행중인 위치
 - Path를 쓰면 / 연산자로 경로를 자연스럽게 이어 붙일 수 있음

```
from pathlib import Path

current_path = Path()
file_path = Path() / "memo.txt"
data_folder_path = Path("data")

print(current_path)      # .
print(file_path)         # memo.txt
print(data_folder_path)  # data
```

파일 경로

- pathlib 라이브러리

```
from pathlib import Path

file_path = Path() / "new_memo.txt"

with open(file_path, "r", encoding="utf-8") as file:
    content = file.read()

print(content)
```

파일 경로

- pathlib 라이브러리
 - 폴더 안의 파일 찾기
 - `glob("*.txt")` : .txt로 끝나는 파일을 찾음

```
from pathlib import Path

data_dir = Path("data")

for file_path in data_dir.glob("*.txt"):
    print(file_path)
```

파일 경로

- pathlib 라이브러리
 - 폴더 위치
 - 터미널에서 실행한 위치 기준

```
data_dir = Path("data")
```

- 현재 파일 기준

```
data_dir = Path(__file__).parent / "data"
```

→ 현재 실행 중인 Python 파일이 있는 폴더 아래의 data 폴더 (같은 위치)

→ 터미널 실행 위치 상관 없이, 특정 Python 파일을 기준으로 상/하위 경로를 잡고 싶을 때 사용

파일 경로

- pathlib 라이브러리 - 현재 파일 기준

path_data_example/

├── data/

| └── stock.txt

└── utils/

├── local_data_loader.py

└── root_data_loader.py

2-3 파일 쓰기

파일 입출력

- 파일 쓰기
 - "memo.txt": 생성할 파일 이름
 - "w": 쓰기 모드
 - encoding="utf-8": 문자 인코딩 방식
 - write(): 파일 내용 쓰기
 - close(): 파일 닫기

```
file = open("memo.txt", "w", encoding="utf-8")  
  
file.write("안녕하세요.\n")  
file.write("Python 파일 입출력을 배우고 있습니다.\n")  
  
file.close()
```

파일 입출력

- `writelines()`
 - `writelines()`는 자동으로 줄바꿈을 넣어주지 않음
 - 줄바꿈이 필요하면 각 문자열에 `\n`을 직접 넣어야 함

```
lines = [  
    "첫 번째 줄\n",  
    "두 번째 줄\n",  
    "세 번째 줄\n"  
]
```

```
with open("data/new_memo.txt", "w", encoding="utf-8") as file:  
    file.writelines(lines)
```

파일 입출력

- `mkdir()`
 - `mkdir()` : 폴더 생성
 - `exist_ok=True` : 이미 폴더가 있어도 오류를 내지 않는 설정 (data 폴더가 있을 때 생략시 에러)

```
from pathlib import Path

data_dir = Path("data")
data_dir.mkdir(exist_ok=True)

file_path = data_dir / "memo.txt"

with open(file_path, "w", encoding="utf-8") as file:
    file.write("data 폴더 안에 파일을 저장합니다.")
```

2-4 CSV

CSV

- Comma-Separated Values
 - 표 데이터를 쉼표로 구분해 저장한 텍스트 파일

name	score	passed
철수	90	True
영희	45	False
민수	80	True

```
name,score,passed  
철수,90,True  
영희,45,False  
민수,80,True
```

CSV

- CSV 파일 생성
 - students.csv 파일 생성
 - 첫 번째 줄은 보통 열 이름, 즉 컬럼명

```
name,score
철수,90
영희,80
민수,70
```

CSV

- CSV 파일 읽기
 - `import csv` : CSV를 다루기 위한 기본 모듈
 - `csv.reader()` : CSV 파일을 한 줄씩 읽음
 - 각 줄은 리스트 형태
 - CSV에서 읽은 값은 기본적으로 문자열

```
import csv

with open("students.csv", "r", encoding="utf-8") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```


CSV

- CSV 파일 읽기
 - `csv.DictReader()` : 첫 줄이 컬럼명일 때 딕셔너리의 키로 자동 변환
 - 리스트의 인덱스가 아닌, 컬럼명으로 값을 조회 가능

```
import csv

with open("students.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row)

# {'name': '철수', 'score': '90'}
# {'name': '영희', 'score': '80'}
# {'name': '민수', 'score': '70'}
```

CSV

- CSV 파일 쓰기

1. `csv.DictWriter(파일 객체, fieldnames)` : 파일 생성 메서드
2. `writeheader()` : CSV 첫 번째 줄에 컬럼 이름을 써주는 메서드
(생략 시 데이터만 저장)
3. `writerows()` : 데이터 추가 메서드

```
import csv

students = [
    {"name": "철수", "score": 90},
    {"name": "영희", "score": 80},
]

with open("students_result.csv", "w", encoding="utf-8", newline="") as file:
    fieldnames = ["name", "score"]

    writer = csv.DictWriter(file, fieldnames=fieldnames)

    writer.writeheader()
    for student in students:
        writer.writerow(student)
```

2-5 JSON

JSON

- JavaScript Object Notation
 - 데이터를 key-value 구조로 저장하는 파일 형식

```
{  
  "name": "Jay",  
  "age": 30,  
  "skills": ["Python", "AI", "Agent"],  
  "is_active": true,  
  "email": null  
}
```

JSON

- JavaScript Object Notation
 - 데이터를 key-value 구조로 저장하는 파일 형식



JSON

- Dictionary vs JSON
 - 데이터를 key-value 구조로 저장하는 파일 형식
 - 딕셔너리(dict) : Python 안에서 사용하는 데이터 타입
 - JSON : 데이터를 저장하거나 주고받기 위한 데이터 포맷

```
{  
    "name": "Jay",  
    "age": 30,  
    "is_active": True,  
    "email": None  
}
```

```
{  
    "name": "Jay",  
    "age": 30,  
    "is_active": true,  
    "email": null  
}
```

JSON

- JSON 파일 쓰기 : `dump()`
 - `json.dump()` : Python 데이터를 JSON 파일로 저장
 - `ensure_ascii=False` : 한글을 그대로 저장하기 위해 사용

```
import json

user = {
    "name": " 홍길동",
    "age": 30,
    "skills": ["Python", "AI", "Agent"],
    "is_active": True
}

with open("user.json", "w", encoding="utf-8") as file:
    json.dump(user, file, ensure_ascii=False, indent=4)
```

JSON

- JSON 파일 읽기 : `load()`
 - `json.load()` : JSON 파일을 읽어서 Python 데이터로 변환
 - JSON object는 Python dict로 변환
 - JSON array는 Python list로 변환

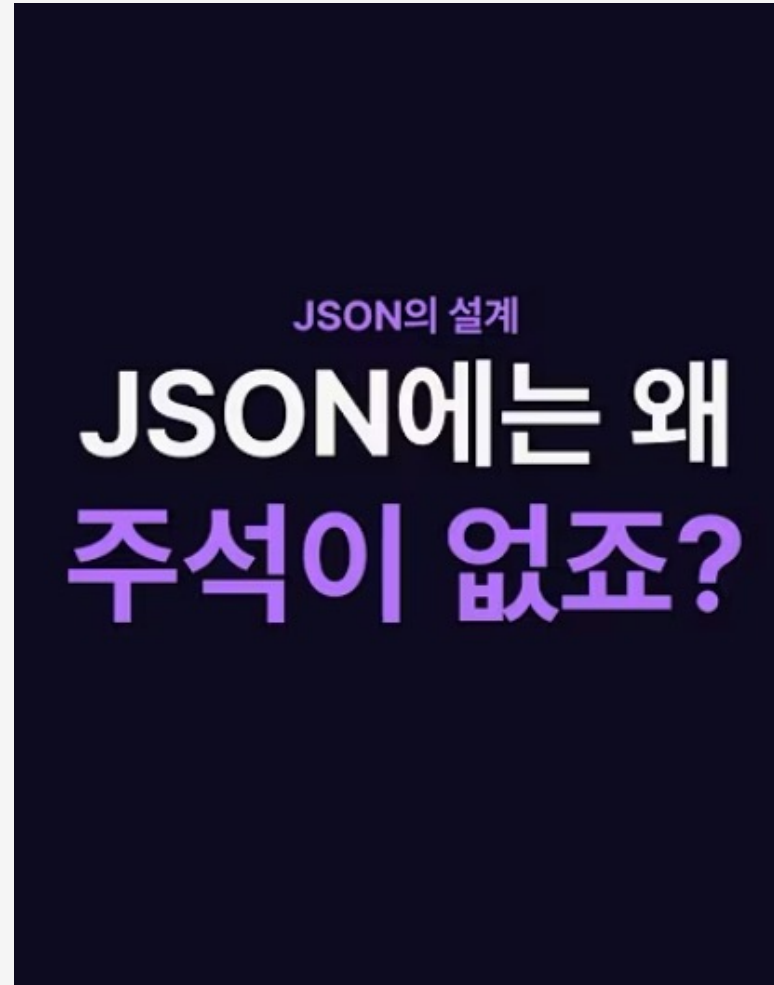
```
import json

with open("user.json", "r", encoding="utf-8") as file:
    user = json.load(file)

print(user)
print(type(user))
print(user["name"])
print(type(user["skills"]))
```


JSON

- JSON은 주석이 없다



제 3장.

예외 처리

3-1 예외 처리

예외 처리

- 프로그램 실행 중 발생한 오류에 대응하는 방법
 - 사용자는 개발자의 의도대로 행동하지 않는다

Dev: creates simple and intuitive UI



User:



예외 처리

- 프로그램 실행 중 발생한 오류에 대응하는 방법
 - 사용자는 개발자의 의도대로 행동하지 않는다

```
age = int(input("나이를 입력하세요: "))  
print(f"내년 나이는 {age + 1}살입니다.")
```

👉 ValueError: invalid literal for int() with base 10: ' 스무 살\x1b[C'

예외 처리

- 프로그램 실행 중 발생한 오류에 대응하는 방법
 - 좋은 프로그램이란

```
age = int(input("나이를 입력하세요: "))  
print(f"내년 나이는 {age + 1}살입니다.")
```

👉 숫자를 입력해야 합니다.
다시 입력해주세요.

try:

오류가 발생할 수 있는 코드

except:

오류가 발생했을 때 실행할 코드

예외 처리

- try-except 기본 구조
 - try 안에는 오류가 날 수 있는 코드를 작성
 - 오류가 발생하면 except 블록이 실행
 - 프로그램이 바로 종료되지 않음

```
try:
    age = int(input("나이를 입력하세요: "))
    print(f"내년 나이는 {age + 1}살입니다.")

except:
    print("숫자만 입력해야 합니다.")
```


try:

오류가 발생할 수 있는 코드

except 특정 오류:

특정 오류가 발생했을 때 실행할 코드

예외 처리

- 특정 예외만 처리하기
 - ValueError : 값이 잘못되었을 때 발생하는 예외

```
try:
    age = int(input("나이를 입력하세요: "))
    print(f"내년 나이는 {age + 1}살입니다.")

except ValueError:
    print("숫자를 입력해야 합니다.")
```

예외 처리

- 예외 종류

예외	발생 상황
ValueError	값의 형태가 잘못되었을 때
TypeError	자료형이 맞지 않을 때
ZeroDivisionError	0으로 나누려고 할 때
FileNotFoundError	파일이 없을 때
KeyError	딕셔너리에 없는 key를 조회할 때
IndexError	리스트에 없는 인덱스를 조회할 때

예외 처리

- 특정 예외 처리하기

```
try:
    age = int(input("나이를 입력하세요: "))
    print(f"내년 나이는 {age + 1}살입니다.")

    result = 100 / age
    print(f"100을 나이로 나눈 값: {result}")

except ValueError :
    print("숫자로 변환할 수 없습니다.")

except ZeroDivisionError :
    print("0으로 나눌 수 없습니다.")
```

예외 처리

- 특정 예외만 처리하기
 - as e : 발생한 예외 정보를 e 변수에 담기
 - 디버깅하거나 로그를 남길 때 유용

```
try:
    age = int(input("나이를 입력하세요: "))
    print(f"내년 나이는 {age + 1}살입니다.")

except ValueError as e:
    print("오류가 발생했습니다.")
    print(e) # invalid literal for int() with base 10: '스무 살'
```

예외 처리

- try-except-else-finally 구조
 - try 성공 → else 실행 → finally 실행
 - try 실패 → except 실행 → finally 실행

try:

오류가 발생할 수 있는 코드

except 예외종류:

오류가 발생했을 때 실행

else:

오류가 발생하지 않았을 때 실행

finally:

항상 마지막에 실행

예외 처리

- 예외가 발생하지 않았을 때 실행 : else
 - try 코드가 정상 실행되면 else가 실행

```
try:
    age = int(input("나이를 입력하세요: "))
    print(f"내년 나이는 {age + 1}살입니다.")

except ValueError as e:
    print("오류가 발생했습니다.")

else:
    print(f"입력한 숫자: {age}")
```

예외 처리

- 항상 실행 : finally
 - 예외 발생 여부와 관계없이 항상 실행
 - 파일 닫기, 연결 종료 등 마무리 작업에 사용

```
try:
    age = int(input("나이를 입력하세요: "))
    print(f"내년 나이는 {age + 1}살입니다.")

except ValueError as e:
    print("오류가 발생했습니다.")

finally:
    print("프로그램을 종료합니다.")
```


예외 처리

- 항상 실행 : finally

```
try:
    file = open("memo.txt", "r", encoding="utf-8")
    content = file.read()
    print(content)

except FileNotFoundError as e:
    print("파일을 찾을 수 없습니다.")
    print(e)

finally:
    if file is not None:
        file.close()
        print("파일을 닫았습니다.")
```

예외 처리

- try-except-else-finally 구조
 - try 성공 → else 실행 → finally 실행
 - try 실패 → except 실행 → finally 실행

try:

오류가 발생할 수 있는 코드

except 예외종류:

오류가 발생했을 때 실행

else:

오류가 발생하지 않았을 때 실행

finally:

항상 마지막에 실행

예외 처리

**예외처리는 오류를 숨기는 것이 아닌,
오류가 났을 때 어떻게 대응할지 정하는 문법**

수고하셨습니다.

